

与其他容器类似, `set` 也是模板。为了定义一个 `set`, 必须指定其元素类型, 本例中是 `string`。与顺序容器类似, 可以对一个关联容器的元素进行列表初始化 (参见 9.2.4 节, 第 300 页)。集合 `exclude` 中保存了 12 个我们想忽略的单词。

此程序与前一个程序的重要不同是, 在统计每个单词出现次数之前, 我们检查单词是否在忽略集合中, 这是在 `if` 语句中完成的:

```
// 只统计不在 exclude 中的单词
if (exclude.find(word) == exclude.end())
```

`find` 调用返回一个迭代器。如果给定关键字在 `set` 中, 迭代器指向该关键字。否则, `find` 返回尾后迭代器。在此程序中, 仅当 `word` 不在 `exclude` 中时我们才更新 `word` 的计数器。

如果用此程序处理与之前相同的输入, 输出将会是:

```
Although occurs 1 time
Before occurs 1 time
are occurs 1 time
as occurs 1 time
...
```

11.1 节练习

练习 11.1: 描述 `map` 和 `vector` 的不同。

练习 11.2: 分别给出最适合使用 `list`、`vector`、`deque`、`map` 以及 `set` 的例子。

练习 11.3: 编写你自己的单词计数程序。

练习 11.4: 扩展你的程序, 忽略大小写和标点。例如, "example.", "example," 和 "Example" 应该递增相同的计数器。

423

11.2 关联容器概述

关联容器 (有序的和无序的) 都支持 9.2 节 (第 294 页) 中介绍的普通容器操作 (列于表 9.2, 第 295 页)。关联容器不支持顺序容器的位置相关的操作, 例如 `push_front` 或 `push_back`。原因是关联容器中元素是根据关键字存储的, 这些操作对关联容器没有意义。而且, 关联容器也不支持构造函数或插入操作这些接受一个元素值和一个数量值的操作。

除了与顺序容器相同的操作之外, 关联容器还支持一些顺序容器不支持的操作 (参见表 11.7, 第 388 页) 和类型别名 (参见表 11.3, 第 381 页)。此外, 无序容器还提供一些用来调整哈希性能的操作, 我们将在 11.4 节 (第 394 页) 中介绍。

关联容器的迭代器都是双向的 (参见 10.5.1 节, 第 365 页)。



11.2.1 定义关联容器

如前所示, 当定义一个 `map` 时, 必须既指明关键字类型又指明值类型; 而定义一个 `set` 时, 只需指明关键字类型, 因为 `set` 中没有值。每个关联容器都定义了一个默认构

造函数，它创建一个指定类型的空容器。我们也可以将关联容器初始化为另一个同类型容器的拷贝，或是从一个值范围来初始化关联容器，只要这些值可以转化为容器所需类型就可以。在新标准下，我们也可以对关联容器进行值初始化：

C++
11

```
map<string, size_t> word_count; // 空容器
// 列表初始化
set<string> exclude = {"the", "but", "and", "or", "an", "a",
                      "The", "But", "And", "Or", "An", "A"};
// 三个元素；authors 将姓映射为名
map<string, string> authors = { {"Joyce", "James"},
                               {"Austen", "Jane"},
                               {"Dickens", "Charles"} };
```

与以往一样，初始化器必须能转换为容器中元素的类型。对于 set，元素类型就是关键字类型。

当初始化一个 map 时，必须提供关键字类型和值类型。我们将每个关键字-值对包围在花括号中：

```
{key, value}
```

来指出它们一起构成了 map 中的一个元素。在每个花括号中，关键字是第一个元素，值是第二个。因此，authors 将姓映射到名，初始化后它包含三个元素。

初始化 multimap 或 multiset

一个 map 或 set 中的关键字必须是唯一的，即，对于一个给定的关键字，只能有一个元素的关键字等于它。容器 multimap 和 multiset 没有此限制，它们都允许多个元素具有相同的关键字。例如，在我们用来统计单词数量的 map 中，每个单词只能有一个元素。另一方面，在一个词典中，一个特定单词则可能具有多个与之关联的词义。

424

下面的例子展示了具有唯一关键字的容器与允许重复关键字的容器之间的区别。首先，我们将创建一个名为 ivec 的保存 int 的 vector，它包含 20 个元素：0 到 9 每个整数有两个拷贝。我们将使用此 vector 初始化一个 set 和一个 multiset：

```
// 定义一个有 20 个元素的 vector，保存 0 到 9 每个整数的两个拷贝
vector<int> ivec;
for (vector<int>::size_type i = 0; i != 10; ++i) {
    ivec.push_back(i);
    ivec.push_back(i); // 每个数重复保存一次
}
// iset 包含来自 ivec 的不重复的元素；miset 包含所有 20 个元素
set<int> iset(ivec.cbegin(), ivec.cend());
multiset<int> miset(ivec.cbegin(), ivec.cend());
cout << ivec.size() << endl;    // 打印出 20
cout << iset.size() << endl;    // 打印出 10
cout << miset.size() << endl;   // 打印出 20
```

即使我们用整个 ivec 容器来初始化 iset，它也只含有 10 个元素：对应 ivec 中每个不同的元素。另一方面，miset 有 20 个元素，与 ivec 中的元素数量一样多。

11.2.1 节练习

练习 11.5: 解释 map 和 set 的区别。你如何选择使用哪个?

练习 11.6: 解释 set 和 list 的区别。你如何选择使用哪个?

练习 11.7: 定义一个 map, 关键字是家庭的姓, 值是一个 vector, 保存家中孩子(们)的名。编写代码, 实现添加新的家庭以及向已有家庭中添加新的孩子。

练习 11.8: 编写一个程序, 在一个 vector 而不是一个 set 中保存不重复的单词。使用 set 的优点是什么?



11.2.2 关键字类型的要求

关联容器对其关键字类型有一些限制。对于无序容器中关键字的要求, 我们将在 11.4 节(第 396 页)中介绍。对于有序容器——map、multimap、set 以及 multiset, 关键字类型必须定义元素比较的方法。默认情况下, 标准库使用关键字类型的<运算符来比较两个关键字。在集合类型中, 关键字类型就是元素类型; 在映射类型中, 关键字类型是元素的第一部分的类型。因此, 11.2 节(第 377 页)中 word_count 的关键字类型是 string。类似的, exclude 的关键字类型也是 string。

425



传递给排序算法的可调用对象(参见 10.3.1 节, 第 344 页)必须满足与关联容器中关键字一样的类型要求。

有序容器的关键字类型

可以向一个算法提供我们自己定义的比较操作(参见 10.3 节, 第 344 页), 与之类似, 也可以提供自己定义的操作来代替关键字上的<运算符。所提供的操作必须在关键字类型上定义一个严格弱序(strict weak ordering)。可以将严格弱序看作“小于等于”, 虽然实际定义的操作可能是一个复杂的函数。无论我们怎样定义比较函数, 它必须具备如下基本性质:

- 两个关键字不能同时“小于等于”对方; 如果 k_1 “小于等于” k_2 , 那么 k_2 绝不能“小于等于” k_1 。
- 如果 k_1 “小于等于” k_2 , 且 k_2 “小于等于” k_3 , 那么 k_1 必须“小于等于” k_3 。
- 如果存在两个关键字, 任何一个都不“小于等于”另一个, 那么我们称这两个关键字是“等价”的。如果 k_1 “等价于” k_2 , 且 k_2 “等价于” k_3 , 那么 k_1 必须“等价于” k_3 。

如果两个关键字是等价的(即, 任何一个都不“小于等于”另一个), 那么容器将它们视作相等来处理。当用作 map 的关键字时, 只能有一个元素与这两个关键字关联, 我们可以用两者中任意一个来访问对应的值。



在实际编程中, 重要的是, 如果一个类型定义了“行为正常”的<运算符, 则它可以用作关键字类型。

使用关键字类型的比较函数

用来组织一个容器中元素的操作的类型也是该容器类型的一部分。为了指定使用自定义的操作, 必须在定义关联容器类型时提供此操作的类型。如前所述, 用尖括号指出要定

义哪种类型的容器，自定义的操作类型必须在尖括号中紧跟着元素类型给出。

在尖括号中出现的每个类型，就仅仅是一个类型而已。当我们创建一个容器（对象）时，才会以构造函数参数的形式提供真正的比较操作（其类型必须与在尖括号中指定的类型相吻合）。

例如，我们不能直接定义一个 `Sales_data` 的 `multiset`，因为 `Sales_data` 没有 `<` 运算符。但是，可以用 10.3.1 节练习（第 345 页）中的 `compareIsbn` 函数来定义一个 `multiset`。此函数在 `Sales_data` 对象的 `ISBN` 成员上定义了一个严格弱序。函数 `compareIsbn` 应该像下面这样定义

```
bool compareIsbn(const Sales_data &lhs, const Sales_data &rhs)
{
    return lhs.isbn() < rhs.isbn();
}
```

426

为了使用自己定义的操作，在定义 `multiset` 时我们必须提供两个类型：关键字类型 `Sales_data`，以及比较操作类型——应该是一种函数指针类型（参见 6.7 节，第 221 页），可以指向 `compareIsbn`。当定义此容器类型的对象时，需要提供想要使用的操作的指针。在本例中，我们提供一个指向 `compareIsbn` 的指针：

```
// bookstore 中多条记录可以有相同的 ISBN
// bookstore 中的元素以 ISBN 的顺序进行排列
multiset<Sales_data, decltype(compareIsbn)*> decltype = decltype type
    bookstore(compareIsbn);
```

此处，我们使用 `decltype` 来指出自定义操作的类型。记住，当用 `decltype` 来获得一个函数指针类型时，必须加上一个 `*` 来指出我们要使用一个给定函数类型的指针（参见 6.7 节，第 223 页）。用 `compareIsbn` 来初始化 `bookstore` 对象，这表示当我们向 `bookstore` 添加元素时，通过调用 `compareIsbn` 来为这些元素排序。即，`bookstore` 中的元素将按它们的 `ISBN` 成员的值排序。可以用 `compareIsbn` 代替 `&compareIsbn` 作为构造函数的参数，因为当我们使用一个函数的名字时，在需要的情况下它会自动转化为一个指针（参见 6.7 节，第 221 页）。当然，使用 `&compareIsbn` 的效果也是一样的。

11.2.2 节练习

练习 11.9: 定义一个 `map`，将单词与一个行号的 `list` 关联，`list` 中保存的是单词所出现的行号。

练习 11.10: 可以定义一个 `vector<int>::iterator` 到 `int` 的 `map` 吗？`list<int>::iterator` 到 `int` 的 `map` 呢？对于两种情况，如果不能，解释为什么。

练习 11.11: 不使用 `decltype` 重新定义 `bookstore`。



11.2.3 pair 类型

在介绍关联容器操作之前，我们需要了解名为 `pair` 的标准库类型，它定义在头文件 `utility` 中。

一个 `pair` 保存两个数据成员。类似容器，`pair` 是一个用来生成特定类型的模板。当创建一个 `pair` 时，我们必须提供两个类型名，`pair` 的数据成员将具有对应的类型。两个类型不要求一样：


```
pair<string, string> anon;           // 保存两个 string
pair<string, size_t> word_count;     // 保存一个 string 和一个 size_t
pair<string, vector<int>> line;      // 保存 string 和 vector<int>
```

427 pair 的默认构造函数对数据成员进行值初始化（参见 3.3.1 节，第 88 页）。因此，anon 是一个包含两个空 string 的 pair，line 保存一个空 string 和一个空 vector。word_count 中的 size_t 成员值为 0，而 string 成员被初始化为空。

我们也可以为每个成员提供初始化器：

```
pair<string, string> author{"James", "Joyce"};
```

这条语句创建一个名为 author 的 pair，两个成员被初始化为"James"和"Joyce"。

与其他标准库类型不同，pair 的数据成员是 public 的（参见 7.2 节，第 240 页）。两个成员分别命名为 first 和 second。我们用普通的成员访问符号（参见 1.5.2 节，第 20 页）来访问它们，例如，在第 375 页的单词计数程序的输出语句中我们就是这么做的：

```
// 打印结果
cout << w.first << " occurs " << w.second
    << ((w.second > 1) ? " times" : " time") << endl;
```

此处，w 是指向 map 中某个元素的引用。map 的元素是 pair。在这条语句中，我们首先打印关键字——元素的 first 成员，接着打印计数器——second 成员。标准库只定义了有限的几个 pair 操作，表 11.2 列出了这些操作。

表 11.2: pair 上的操作	
pair<T1, T2> p;	p 是一个 pair，两个类型分别为 T1 和 T2 的成员都进行了值初始化（参见 3.3.1 节，第 88 页）
pair<T1, T2> p(v1, v2)	p 是一个成员类型为 T1 和 T2 的 pair；first 和 second 成员分别用 v1 和 v2 进行初始化
pair<T1,T2>p = {v1,v2} ;	等价于 p (v1,v2)
make_pair(v1, v2)	返回一个用 v1 和 v2 初始化的 pair。pair 的类型从 v1 和 v2 的类型推断出来
p.first	返回 p 的名为 first 的（公有）数据成员
p.second	返回 p 的名为 second 的（公有）数据成员
p1 relop p2	关系运算符 (<, >, <=, >=) 按字典序定义：例如，当 p1.first < p2.first 或!(p2.first < p1.first) && p1.second < p2.second 成立时，p1 < p2 为 true。关系运算利用元素的<运算符来实现
p1 == p2	当 first 和 second 成员分别相等时，两个 pair 相等。相等性判断利用元素的==运算符实现
p1 != p2	

创建 pair 对象的函数

C++ 11 想象有一个函数需要返回一个 pair。在新标准下，我们可以对返回值进行列表初始化（参见 6.3.2 节，第 203 页）

```
428 pair<string, int>
    process(vector<string> &v)
    {
        // 处理 v
```

```
    if (!v.empty())
        return {v.back(), v.back().size()}; // 列表初始化
    else
        return pair<string, int>(); // 隐式构造返回值
}
```

若 `v` 不为空，我们返回一个由 `v` 中最后一个 `string` 及其大小组成的 `pair`。否则，隐式构造一个空 `pair`，并返回它。

在较早的 C++ 版本中，不允许用花括号包围的初始化器来返回 `pair` 这种类型的对象，必须显式构造返回值：

```
    if (!v.empty())
        return pair<string, int>(v.back(), v.back().size());
```

我们还可以用 `make_pair` 来生成 `pair` 对象，`pair` 的两个类型来自于 `make_pair` 的参数：

```
    if (!v.empty())
        return make_pair(v.back(), v.back().size());
```

11.2.3 节练习

练习 11.12: 编写程序，读入 `string` 和 `int` 的序列，将每个 `string` 和 `int` 存入一个 `pair` 中，`pair` 保存在一个 `vector` 中。

练习 11.13: 在上一题的程序中，至少有三种创建 `pair` 的方法。编写此程序的三个版本，分别采用不同的方法创建 `pair`。解释你认为哪种形式最易于编写和理解，为什么？

练习 11.14: 扩展你在 11.2.1 节练习（第 378 页）中编写的孩子姓到名的 `map`，添加一个 `pair` 的 `vector`，保存孩子的名和生日。

11.3 关联容器操作

除了表 9.2（第 295 页）中列出的类型，关联容器还定义了表 11.3 中列出的类型。这些类型表示容器关键字和值的类型。

表 11.3: 关联容器额外的类型别名	
<code>key_type</code>	此容器类型的关键字类型
<code>mapped_type</code>	每个关键字关联的类型；只适用于 <code>map</code>
<code>value_type</code>	对于 <code>set</code> ，与 <code>key_type</code> 相同 对于 <code>map</code> ，为 <code>pair<const key_type, mapped_type></code>

对于 `set` 类型，**`key_type`** 和 **`value_type`** 是一样的；`set` 中保存的值就是关键字。在一个 `map` 中，元素是关键字-值对。即，每个元素是一个 `pair` 对象，包含一个关键字和一个关联的值。由于我们不能改变一个元素的关键字，因此这些 `pair` 的关键字部分是 `const` 的：

```
set<string>::value_type v1; // v1 是一个 string
set<string>::key_type v2;  // v2 是一个 string
map<string, int>::value_type v3; // v3 是一个 pair<const string, int>
map<string, int>::key_type v4;  // v4 是一个 string
map<string, int>::mapped_type v5; // v5 是一个 int
```